

Operations Research

Traveling Salesperson Problem

Prof. Dr. Martin Bichler

Department of Computer Science

School of Computation, Information, and Technology

Technical University of Munich

Course outline

- **Linear Optimization**

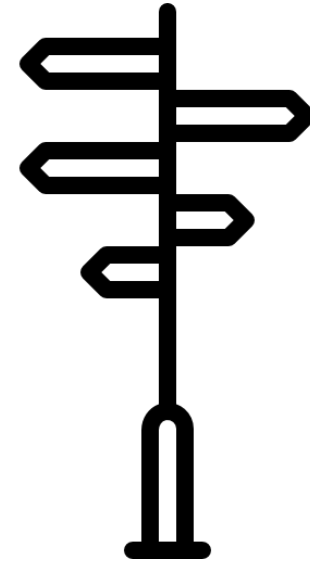
- Modeling by linear programming and graphical solution
- Solving linear programs, convexity
- Simplex algorithm, 2-phase simplex
- Simplex algorithm in matrix notation
- Sensitivity analysis, re-optimization
- Duality theory, zero-sum games

- **Integer Linear Optimization**

- Modeling integer programming problems (IPs)
- Solving IPs: branch & bound, cutting planes
- Advanced solution methods: column generation, approximation algorithms
- Fast solvable integer problems: total unimodularity, matroids
- Network flow problems
- **Traveling salesperson problem**

- **Nonlinear Optimization**

- Optimization of multidimensional functions
- Convex optimization

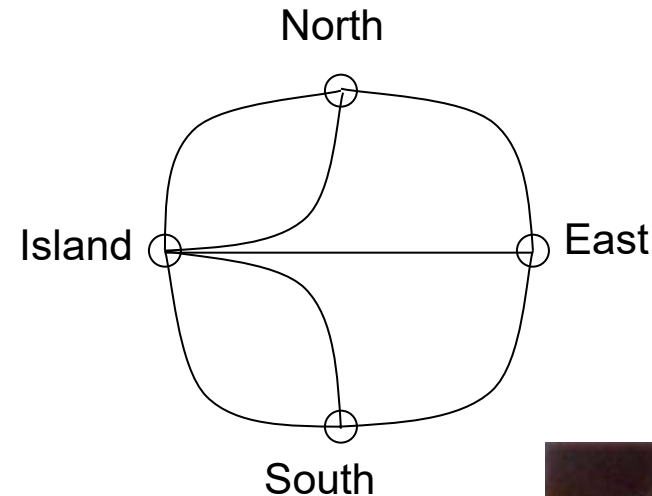
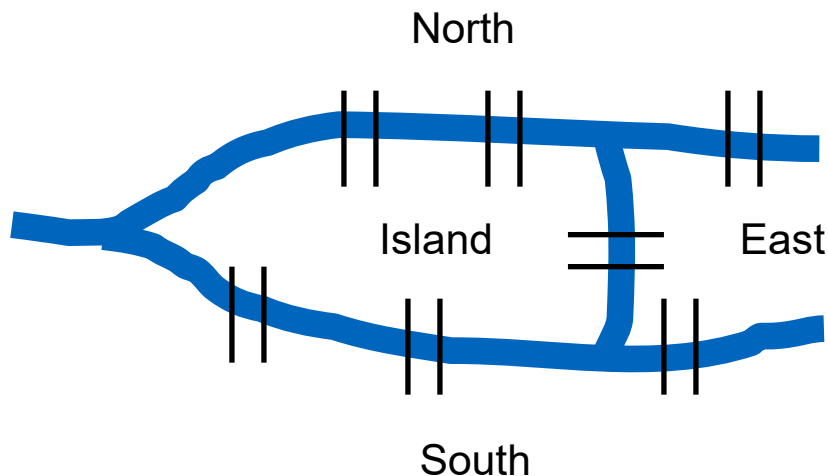


Today you will learn ...

- how to efficiently solve simple tour planning problems (**Euler tours**).
- how to model and solve the **Traveling Salesperson Problem** (TSP) exactly.
- about **approximation algorithms** and heuristics for the TSP.



The classic tour planning: The Königsberg bridge problem by Euler, 1736



Problem:

Finding a tour around Königsberg, crossing all bridges exactly once.

Modeling (abstraction) by neglecting unessential details and restricting to essential information: bridges and districts.



Leonhard Euler
Source: wikipedia.org

Route planning

Königsberg bridge problem:

- Finding a tour in which each bridge (edge) is crossed exactly once.
- Application in optimal scheduling e.g., street cleaning, winter service, garbage collection in a traffic network where each road has to be passed only once.

Definition: Eulerian path, cycle, and graph

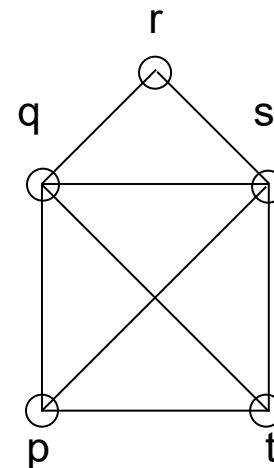
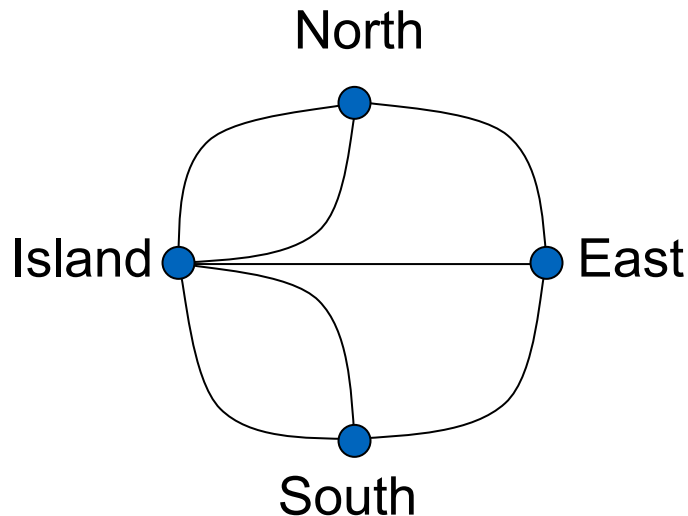
- An **Eulerian path** is a path that contains each edge of a graph exactly once.
- If start point = end point, then it is an **Eulerian cycle**.
- A graph is called **Eulerian** if it contains an Eulerian cycle.

Eulerian path

Definition: The **degree** of a node v is the number of edges to which v is incident.

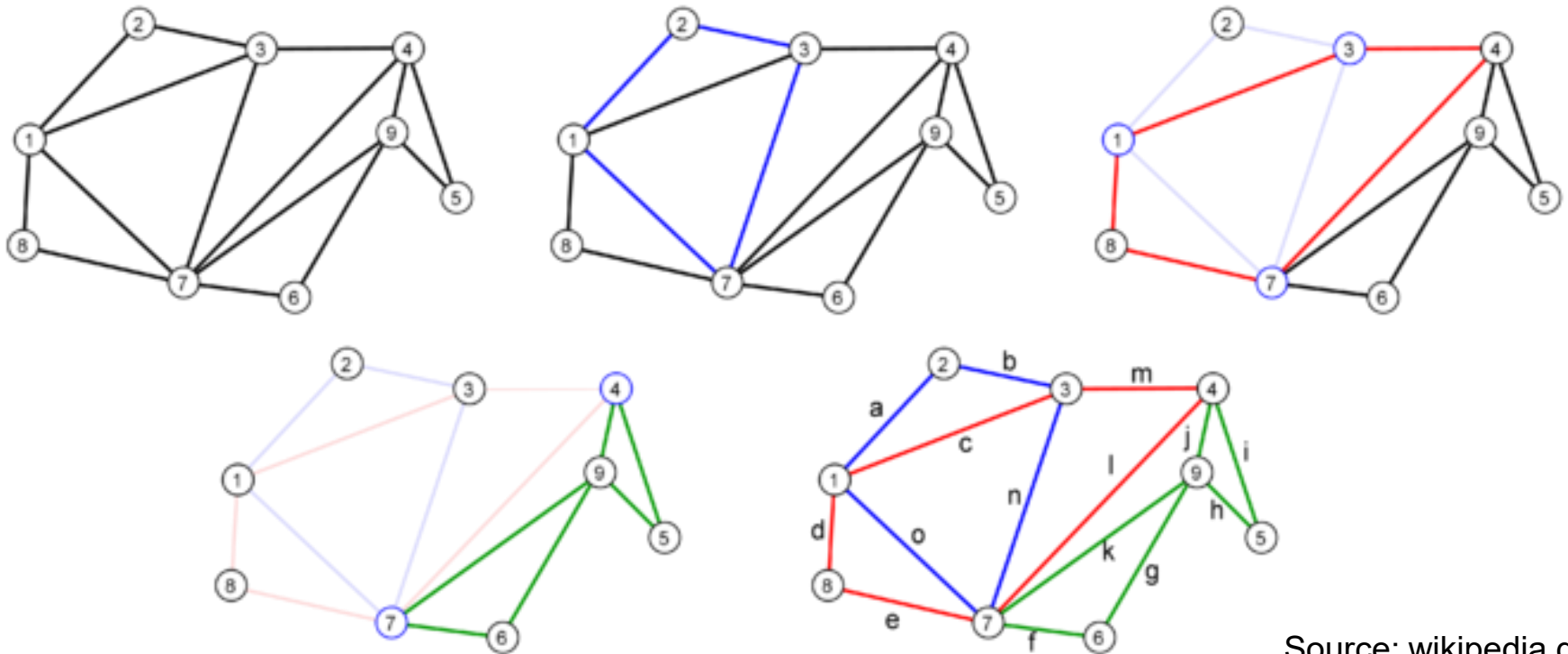
Euler discovered the following law:

- In a connected graph there is at least one Euler path (Euler tour), if this has at most 2 nodes with odd degree.
- If no node has odd degree, the Eulerian path is an Eulerian cycle.



Algorithm by Hierholzer (1873)

To find an Eulerian cycle (if one exists), there is a simple linear-time algorithm that identifies cycles one by one and then assembles them.



Source: wikipedia.de

Eulerian graph

A graph G is called **Eulerian**, if there is a cycle in G which passes through each edge of G exactly once.

Theorem:

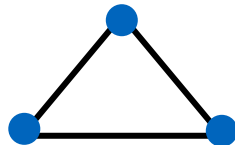
G is *Eulerian* if and only if G is connected and each node has even degree.

Proof:

$\Rightarrow$: If there is an Eulerian cycle, all degrees are even. Clear.

$\Leftarrow$: If all degrees are even, there is an Eulerian cycle.

- Show by induction over the number of edges
- 3 edges:



$\rightarrow$ Eulerian cycle

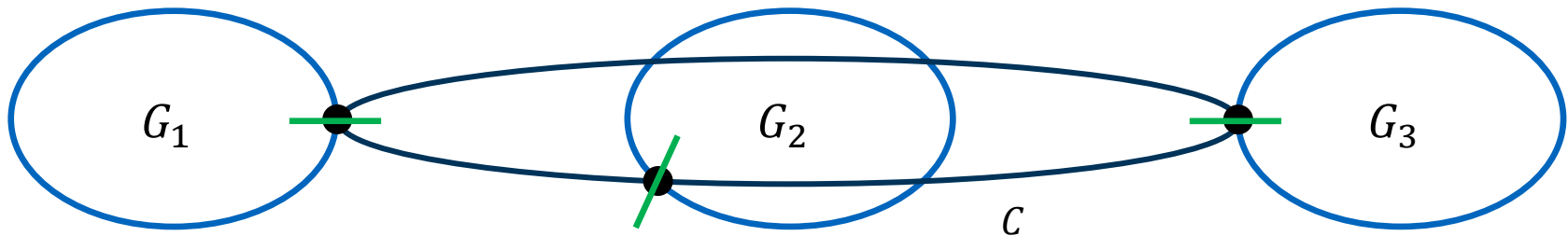
- $k \geq 3$ edges: Since all nodes have degree ≥ 2 , there is a cycle C in G .
- Start at any node and move along any edges until a node is visited for the second time.

Eulerian graph

Proof (continued):

Let $G_1, \dots, G_k$ be the subgraphs of G after removing the edges in $\mathcal{C}$. Since $\mathcal{C}$ is a cycle, the node degrees in each G_i are even. Thus, by induction, there is an Eulerian cycle in each G_i .

These Euler cycles and cycle $\mathcal{C}$ can be easily combined to form an Eulerian cycle of G .



Hamiltonian cycle

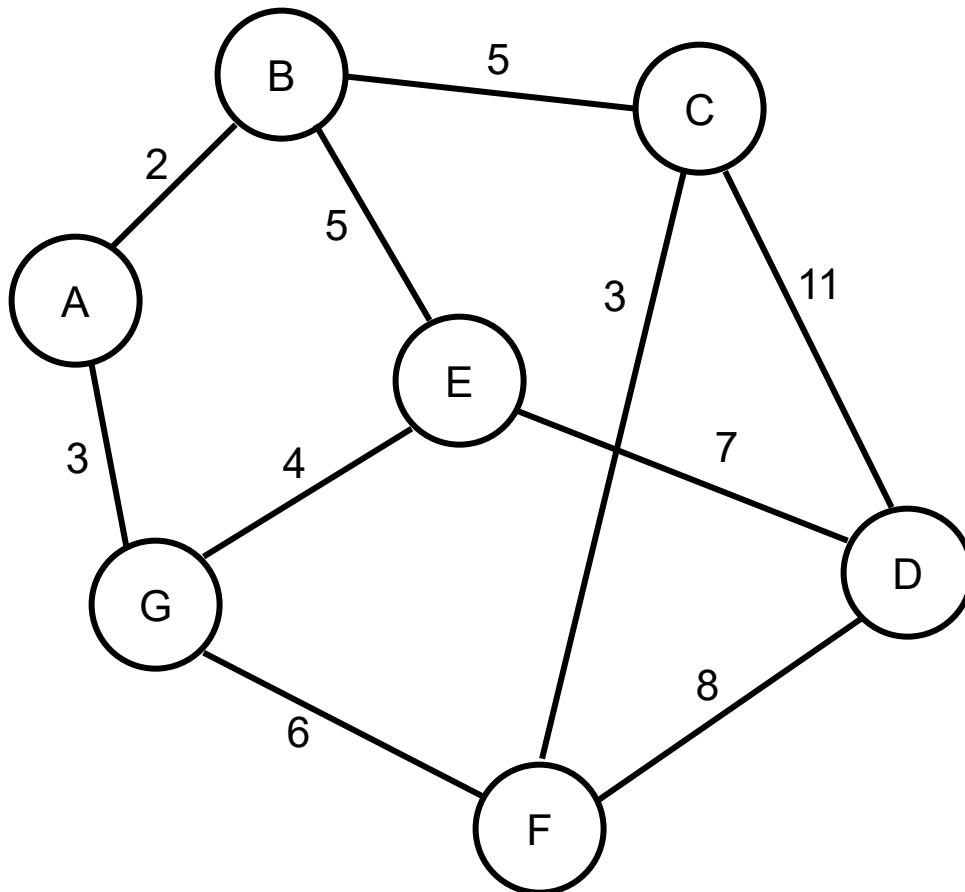
A **Hamiltonian cycle** of a graph G is a cycle that contains each node of G exactly once.

A graph G is called **Hamiltonian** if it contains a Hamiltonian cycle.

The problem of finding a Hamiltonian cycle (or tour) in a *weighted* graph is called the **traveling salesperson problem (TSP)**.

- **Asymmetric TSP**: Outward and return paths have different lengths.
- **Metric TSP**: Symmetric TSP and the triangle inequality is fulfilled, i.e. the direct connection between 2 nodes is never longer than a detour ($c_{ij} \leq c_{ik} + c_{kj}$).

Traveling Salesperson Problem (TSP)



A traveling salesperson must visit all n cities exactly once to reach his home city again. Which route minimizes the total duration of the trip?

Complete search is in $O(n!)$

Proof of **NP**-hardness
by Karp 1972

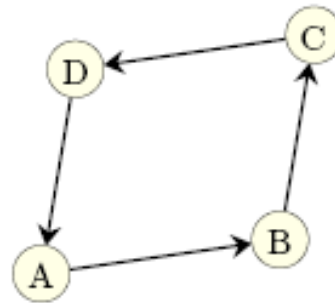
Number of round trips through 4 cities

Asymmetrical TSP:

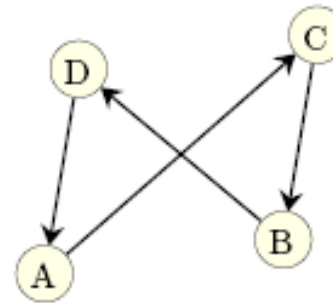
$$(n - 1)!$$

Symmetrical TSP:

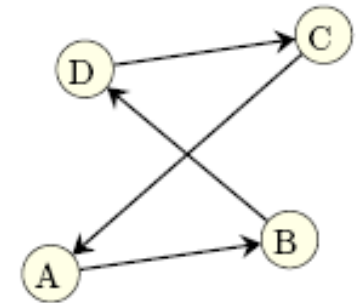
$$\frac{(n - 1)!}{2}$$



1: A-B-C-D-A

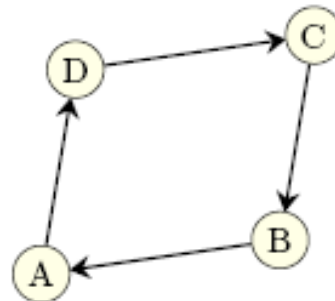


2: A-C-B-D-A

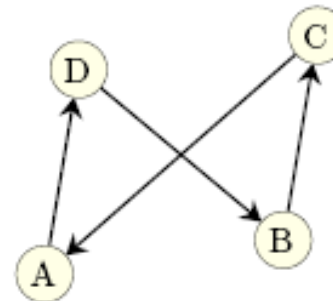


3: A-B-D-C-A

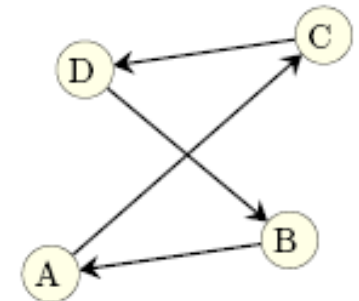
# cities	# tours (asym.)
3	2
4	6
10	362.800
49 (USA)	ca. 10^{61}
24978 (SE)	ca. 10^{98992}



4: A-D-C-B-A



5: A-D-B-C-A



6: A-C-D-B-A

Source: S. Näher, Algorithmus der Woche

Solution sizes (for exact solutions)

1954
n=49

1977
n=120

1987
n=666

1987
n=2392

1994
n=7397

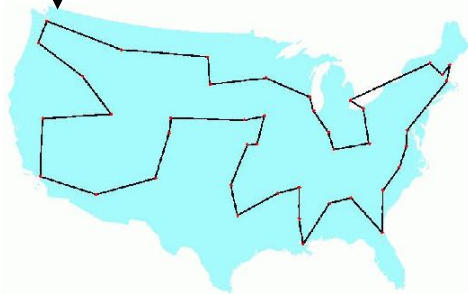
1998
n=13509

2001
n=15112

2004
n=24978

2017
n=109.399

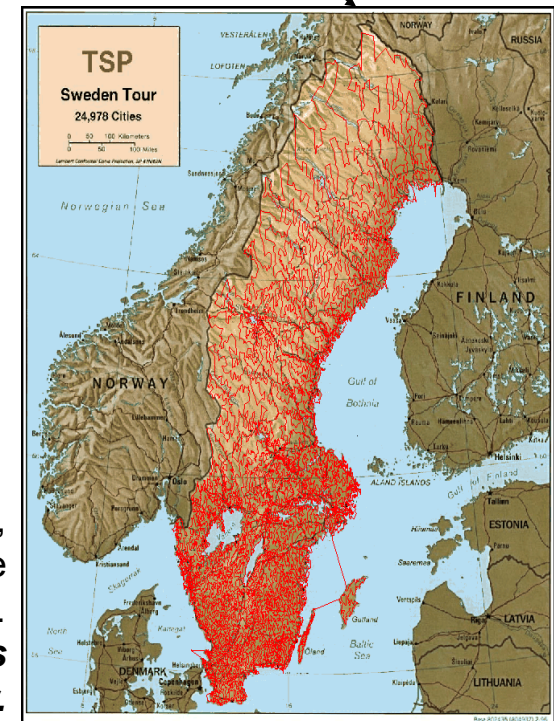
1954: Dantzig, Fulkerson, and Johnson found the optimal tour through 49 U.S. cities.



2019:
n=2.079.471
almost optimally solved
(0.0000074 MIP gap)
2.162.098.780.185
variables



Applegate, Bixby, Chvátal, Cook, and Helsgaun (2004) found the optimal tour through 24,978 cities in Sweden. The computation time was **84.8 CPU years on an Intel Xeon 2.8 GHz processor.**



Modeling as IP

Given:

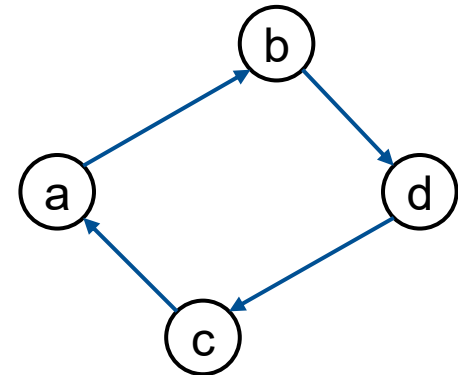
- n locations $i \in N = \{1, \dots, n\}$
- distances c_{ij} between location i and location j

Wanted:

- Tour through all locations

Decision variable:

- x_{ij} : Use path from city i to city j
(either for all $i, j \in N$ if a completely directed graph is given, otherwise for all connections that exist)



$$x_{ab} = x_{bd} = x_{dc} = x_{ca} = 1$$

and e.g. $x_{ad} = x_{ac} = x_{ba} = 0$

Traveling Salesperson Problem as IP

Constraints:

- Each city must be arrived at exactly once.

$$\sum_{i \in N \setminus \{j\}} x_{ij} = 1 \quad \text{for each } j \in N$$

- Each city must be left exactly once.

$$\sum_{j \in N \setminus \{i\}} x_{ij} = 1 \quad \text{for each } i \in N$$

Problems?

$$\begin{array}{ll} \min & \sum_{i \in N} \sum_{j \in N} c_{ij} x_{ij} \\ \text{s.t.} & \sum_{i \in N \setminus \{j\}} x_{ij} = 1 \quad \text{for each } j \in N \\ & \sum_{j \in N \setminus \{i\}} x_{ij} = 1 \quad \text{for each } i \in N \\ & x_{ij} \in \{0,1\} \end{array}$$

What is still missing in the formulation: The problem of subtours

Eliminate 2-cities tours:

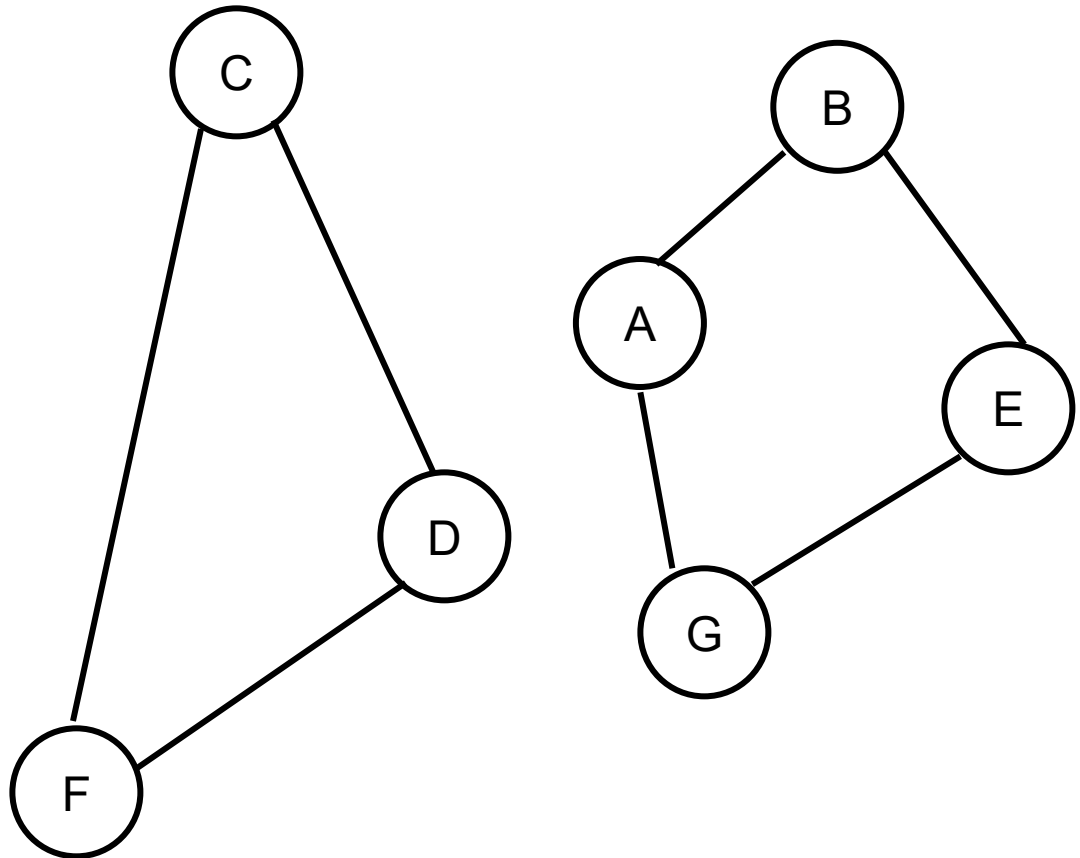
$$x_{ij} + x_{ji} \leq 1$$

Eliminate 3-cities tours:

$$x_{ij} + x_{jk} + x_{ki} \leq 2$$

Eliminate tours visiting
cities in set U :

$$\sum_{i \in U} \sum_{j \in U} x_{ij} \leq |U| - 1$$



TSP as IP

$$\begin{array}{ll}\min & \sum_{i \in N} \sum_{j \in N} c_{ij} x_{ij} \\ \text{s.t.} & \sum_{i \in N \setminus \{j\}} x_{ij} = 1 \quad \text{for each } j \in N \\ & \sum_{j \in N \setminus \{i\}} x_{ij} = 1 \quad \text{for each } i \in N \\ & \sum_{i \in U} \sum_{j \in U} x_{ij} \leq |U| - 1 \quad \text{for each } U \in N, 2 < U < N - 1 \\ & x_{ij} \in \{0, 1\}\end{array}$$

- formulation goes back to Dantzig, Fulkerson and Johnson (1954), for the solution of the TSP across the USA with 49 nodes
- commonly known as Subtour Elimination Constraints (SEC)
- SEC are used in the fastest TSP solvers today (such as Concorde)
<https://www.math.uwaterloo.ca/tsp/>

Alternative elimination of the subtours

Idea: Introduce a label $u_i \in [1, \dots, n]$ for each city $i \in N$ and ensure that all cities in a tour have ascending labels; only city with label 1 may follow city with label n .

Constraints by Miller, Tucker and Zemlin (MTZ, 1960):

$$u_1 = 1$$

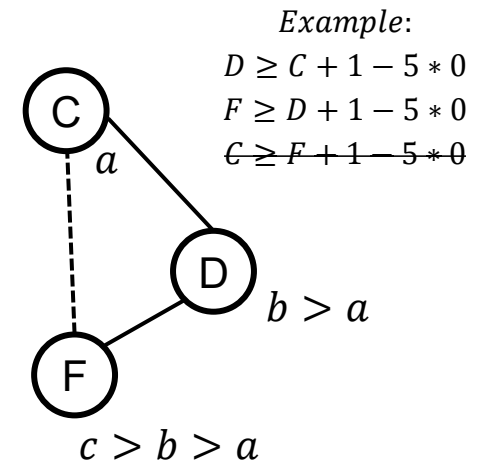
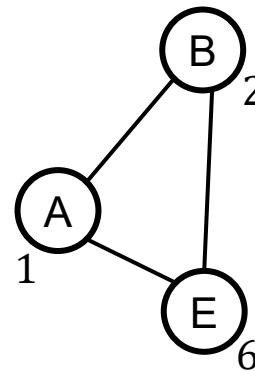
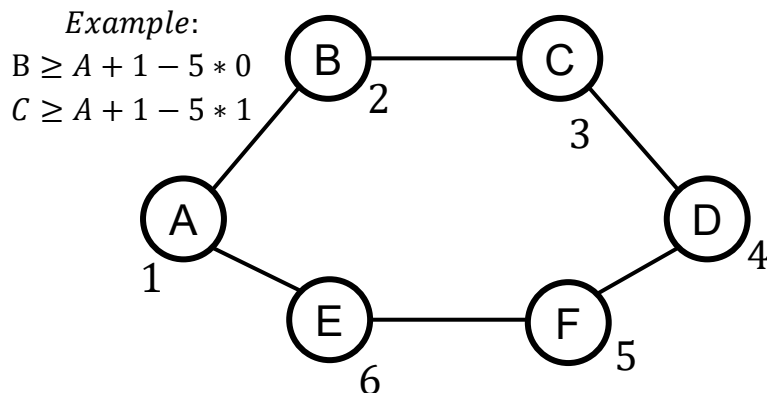
(city 1 gets label 1)

$$2 \leq u_i \leq n$$

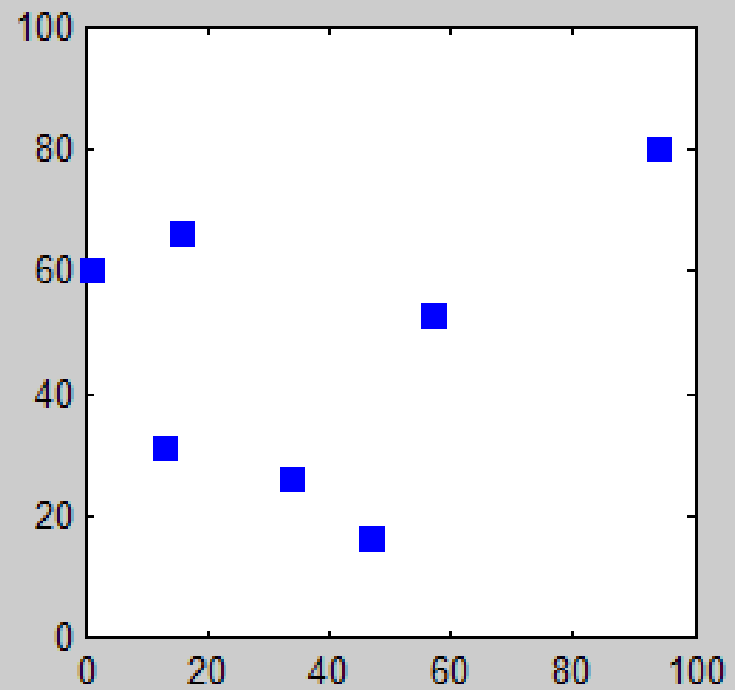
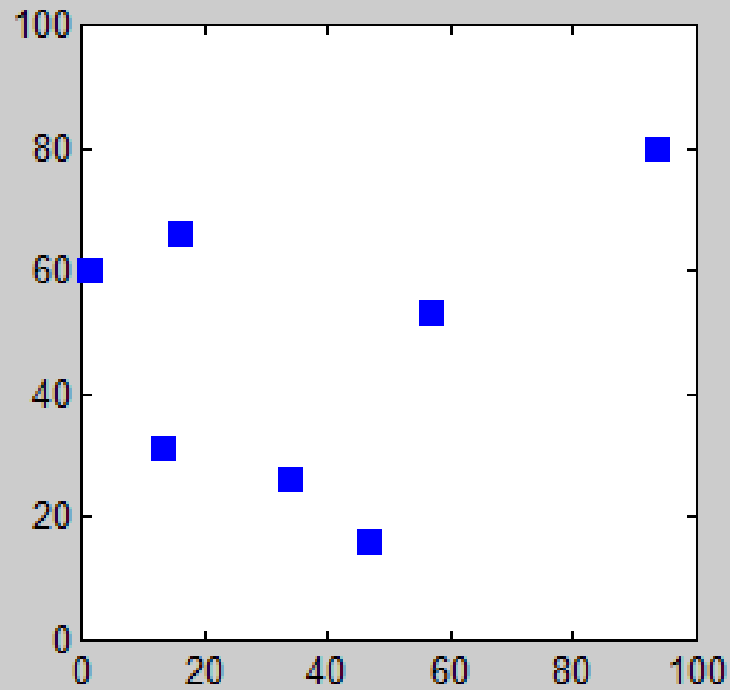
for each $i \in N \setminus \{1\}$

$$u_j \geq u_i + 1 - (n - 1)(1 - x_{ij})$$

for each $i, j \in N \setminus \{1\}, i \neq j // u_j$ is 1 larger than u_i



Animation Branch & Bound



Source: https://en.wikipedia.org/wiki/Travelling_salesman_problem

Comparison between SEC and MTZ

SEC:

- exponentially many constraints
- provides very good bounds (strong relaxation!)

MTZ:

- only quadratic many constraints
- provides very bad bounds (weak relaxation!)

Idea:

- create SEC constraints one by one
→ Branch & Cut Algorithm

B&B using the assignment problems

Alternatively, the assignment problems which lead to a cycle can be solved iteratively to set binary variables to 0 using a B&B approach.

Solving the assignment problem using the Hungarian algorithm.

Example:

Solution of the assignment problem S_1 results in $x_{15} = x_{21} = x_{34} = x_{43} = x_{52} = 1$, $z = 495$

	C1	C2	C3	C4	C5
C1	M	132	217	164	58
C2	132	M	290	201	79
C3	217	290	M	113	303
C4	164	201	113	M	196
C5	58	79	303	196	M



	C1	C2	C3	C4	C5
C1	M	132	217	164	58
C2	132	M	290	201	79
C3	217	290	M	M	303
C4	164	201	113	M	196
C5	58	79	303	196	M

B&B using the assignment problems

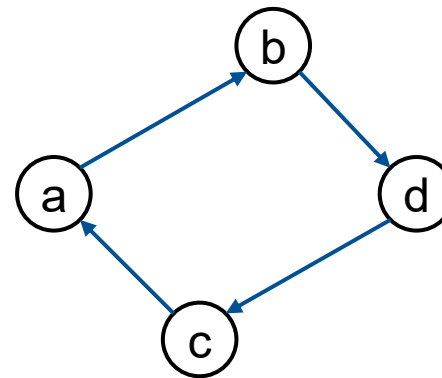
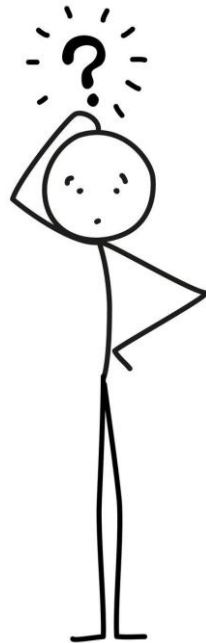
Alternatively, the assignment problems which lead to a cycle can be solved iteratively to set binary variables to 0 using a B&B approach.

Solving the assignment problem using the Hungarian algorithm.

Example:

- Solution of the assignment problem S_1 results in $x_{15} = x_{21} = x_{34} = x_{43} = x_{52} = 1$, $z = 495$
- This solution contains the subtours $1 - 5 - 2 - 1$ and $3 - 4 - 3$
- Branching by adding the following subproblems, where some costs are set to M :
 - $S_2: S_1$ and $x_{34} = 0 \Rightarrow$ results in 2 subtours and $z = 652$
 - $S_4: S_2$ and $x_{25} = 0 \Rightarrow$ no subtour and $z = 668$ (incumbent)
 - $S_5: S_2$ and $x_{52} = 0 \Rightarrow$ 2 subtours and $z = 704$ (stop, because > 668)
 - $S_3: S_1$ and $x_{43} = 0 \Rightarrow$ results in 2 subtours and $z = 652$
 - $S_6: S_3$ and $x_{25} = 0 \Rightarrow$ no subtour and $z = 704$ (stop, because > 668)
 - $S_7: S_3$ and $x_{52} = 0 \Rightarrow$ 2 subtours and $z = 910$ (stop, because > 668)

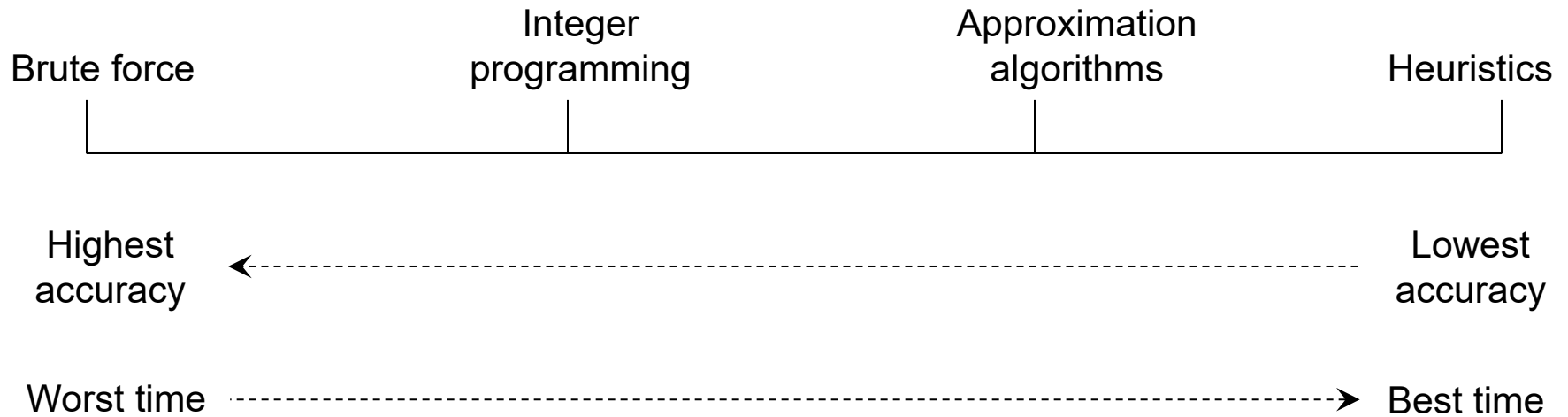
Write down the MTZ constraints for the network below.



Heuristics and approximation for TSP

How do we get the best possible (initial) solution for the TSP?

- Heuristics
- Improvement methods
- Approximation algorithms



Nearest Neighbor-Heuristic

Start with any node $v = v_0$.

Set of nodes already visited $V' = v_0$.

While ($V' \neq V$) {

- Find the node $v' \in V \setminus V'$ with smallest distance to v .
- Add edge (v, v') to the tour.
- $v = v', V' = V' \cup \{v'\}$

}

Add edge (v, v_0) to the tour.

The last edge can be arbitrarily far away from the start.

→ No approximation with constant factor is possible.

→ Reasonable average solution quality with a fast runtime $O(n^2)$.

Improvement procedure (local search)

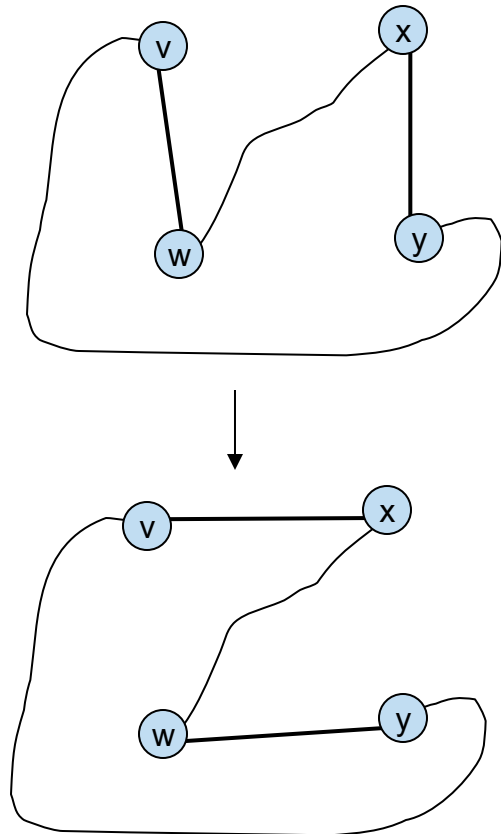
Improving optimization methods try to shorten an existing tour (e.g. from a heuristic) by small modifications.

If none of the considered modifications leads to an improvement, a local optimum is found (but not necessarily a global optimum).

Example: 2-Opt

Take 2 edges (v,w) and (x,y) and replace them by (v,x) and (w,y) OR (v,y) and (w,x) to get a tour again (depending on whether it results in a Hamilton cycle again).

- no approximation guarantee
- very good average results, often close to OPT



MST-doubling algorithm for metric TSP

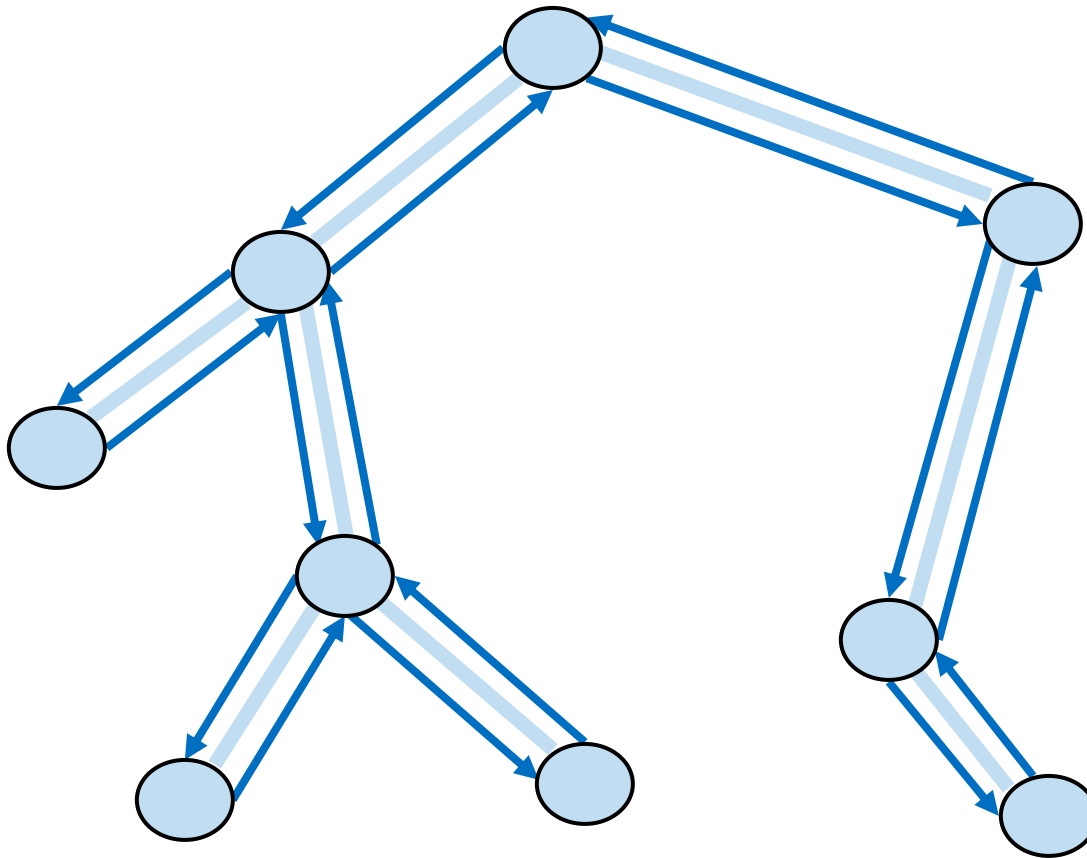
The algorithm Rosenkratz, Steans, and Lewis (1977) computes an approximation of the metric and symmetric TSP. The original version is a 2-approximation, an improved version yields a 1.5-approximation, which was for a long time the best approximation quality for the TSP.

There is an undirected weighted graph $G = (V, E)$.
Calculate a minimum **spanning tree** $T' = (V, E')$.

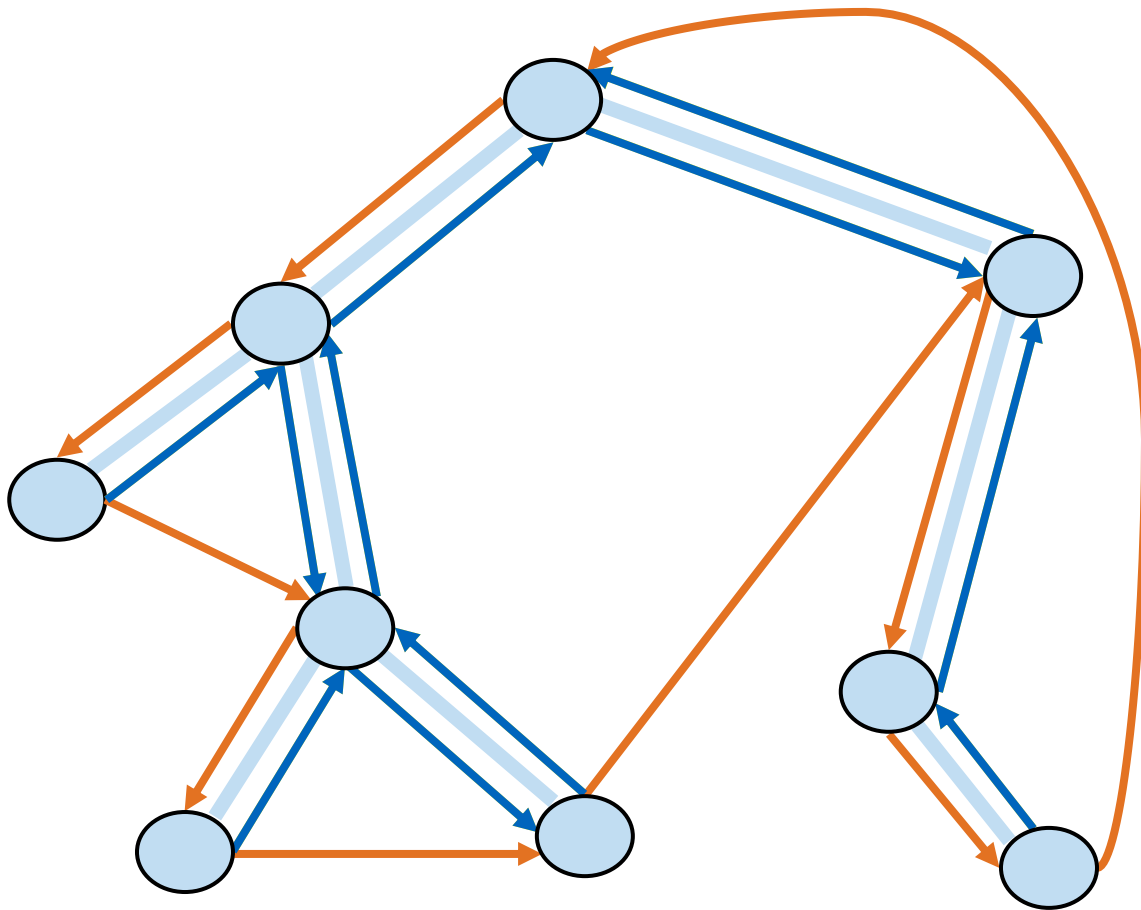
- Construct an Eulerian tour on T' .
- As long as T' is not a TSP tour:

Travel the Euler tour, and whenever a node would be visited for the second time, skip that node and cut to the successor node.

Minimum spanning tree



Second step in the MST-doubling algorithm



Approximation ratio of the MST-doubling algorithm

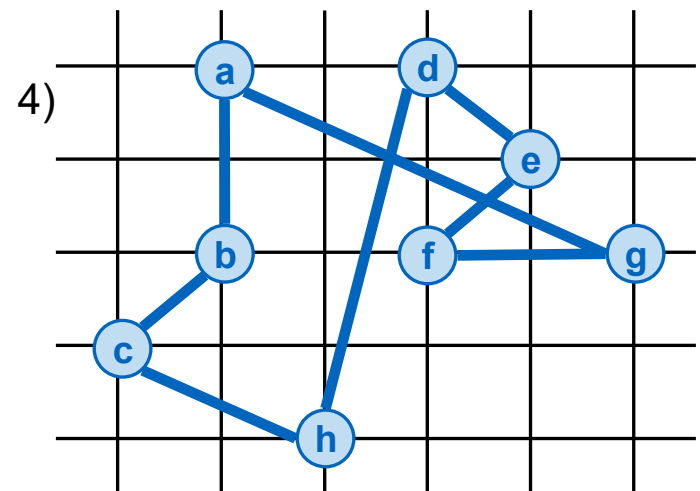
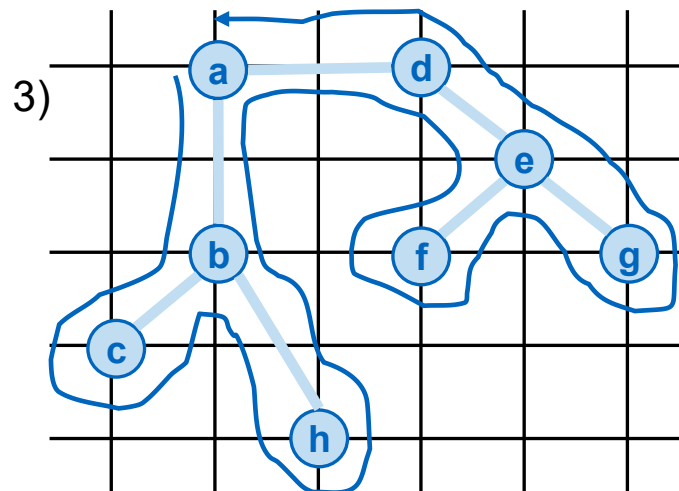
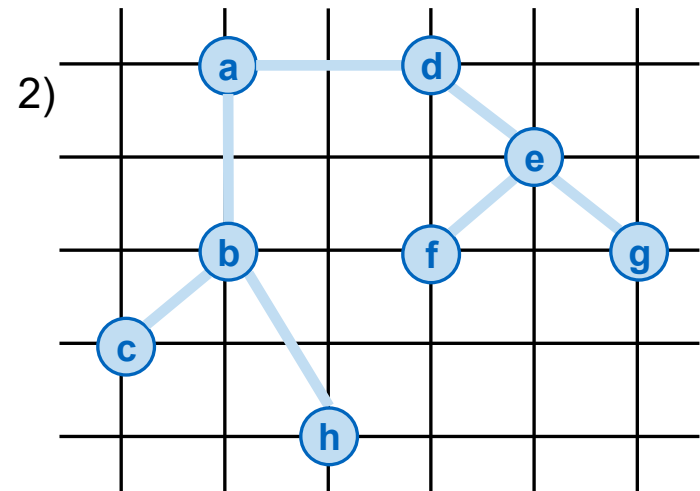
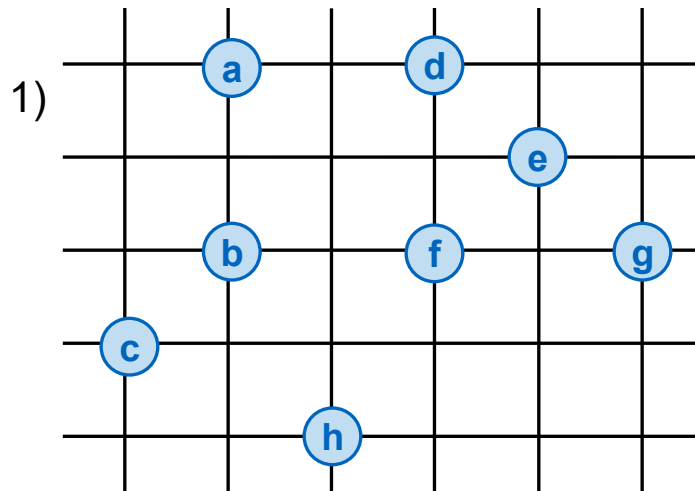
Theorem:

The MST-doubling algorithm yields a 2-approximation for TSP (in a metric graph satisfying the triangle inequality).

Proof:

- OPT is the optimal solution (set of edges) of the TSP.
 - T is the MST.
 - A is the set of edges chosen by the algorithm.
 - EC is the set of edges in the Eulerian circle.
-
- $\text{cost}(T) \leq \text{cost}(\text{OPT})$
 - $\text{cost}(\text{EC}) = 2\text{cost}(T) \leq 2\text{cost}(\text{OPT})$.
 - OPT is a cycle, removing one edge is a spanning tree
 - $\text{cost}(A) \leq \text{cost}(\text{EC})$
 - A is created by taking shortcuts in EC.
- $\Rightarrow \text{cost}(A) \leq 2\text{cost}(\text{OPT})$

Steps in the MST-doubling algorithm

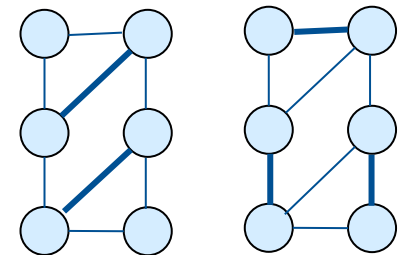


Detour: what is a matching in a graph?

Consider a set of items (nodes) and available information about which item can be assigned to another one (edges).

A **matching** is a set of edges in a graph that do not have any nodes in common.

- **The Maximum Cardinality Matching Problem:** How to find a matching that matches the maximum number of items?
- **Perfect Matching:** Is there a matching that matches all items? Every vertex is incident to exactly one selected edge.
- **Bipartite Matching:** Suppose we have exactly two classes of items, and suppose we know that there were possible matchings exclusively between those classes. Find a max. weight matching between the two classes is the assignment problem.



Handshaking lemma

Lemma: Let $G = (V, E)$ be any undirected graph. Then $\sum_{v \in V} \deg(v) = 2|E|$

If you add up the degrees of all vertices, every edge is counted exactly twice, once at each endpoint. Hence, the total degree sum must be even.

Corollary: The number of odd vertices in G is even.

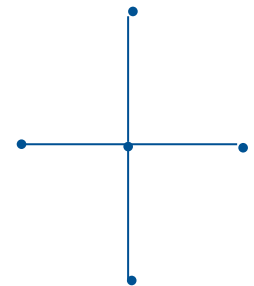
Let's divide the vertex set V into V_{even} and V_{odd}

The total degree sum $\sum_{v \in V_{\text{even}}} \deg(v) + \sum_{v \in V_{\text{odd}}} \deg(v) = 2|E|$, which is even

The sum over $\sum_{v \in V_{\text{even}}} \deg(v)$ is even, bc. It is a sum of even numbers.

The sum over $\sum_{v \in V_{\text{odd}}} \deg(v)$ is even, bc. „even = even + odd“

It follows that the number of odd-degree vertices is even.



Example w. 4 odd degree vertices

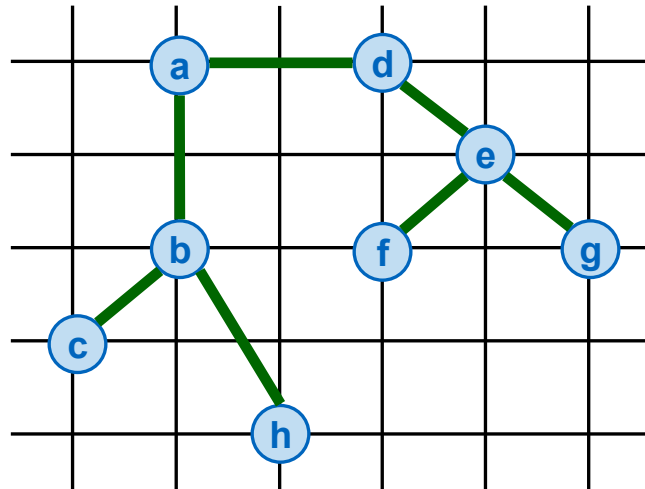
Improvement by matching

Nicos Christofides extended the 2-approximation and published a method that gives a 1.5-approximation in the worst case:

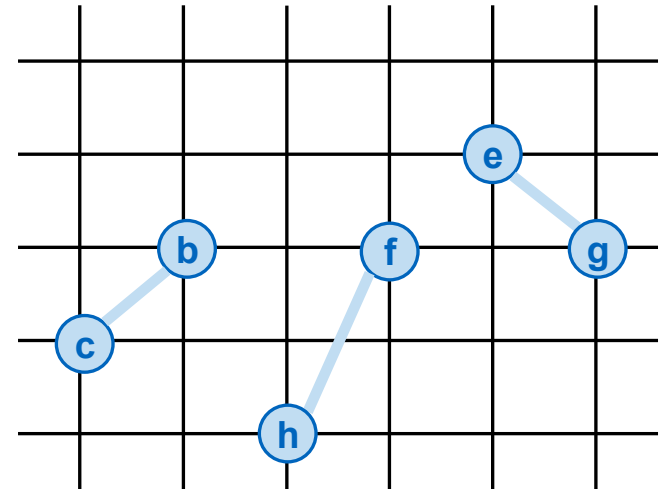
1. Calculate the MST graph T .
2. Compute a matching with minimum weight (M) between all nodes with odd degree.
3. Combine T and M as edge sets into a multigraph and compute an Eulerian cycle.
4. Traverse all nodes and take shortcuts whenever a node has already been visited. (Because the weight function is metric, the path $a - c$ is always shorter than the one via $a - b - c$.)

Improvement by matching

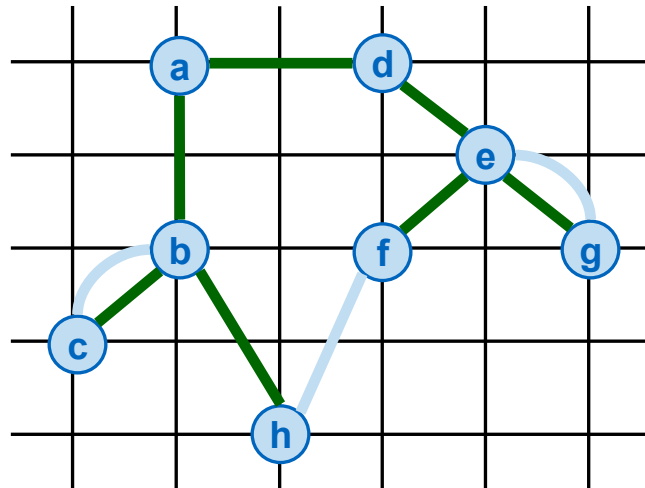
1)
MST T



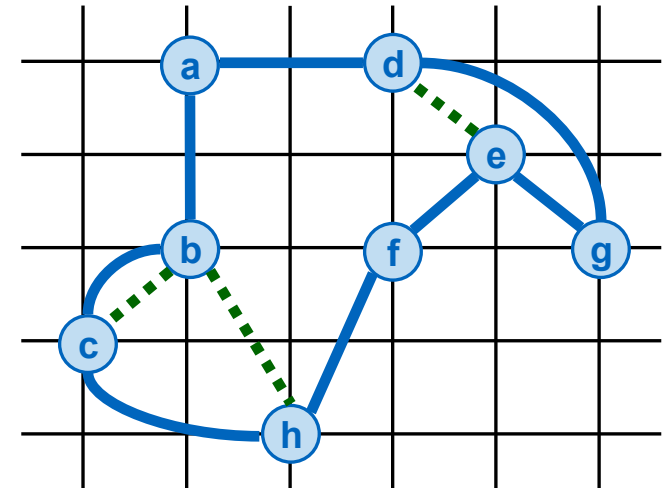
2)
Matching M
on nodes
with odd
degree



3)
 $T \cup M$



4)
Euler tour on
 $T \cup M$. Then
iteratively
remove
reoccurring
nodes.



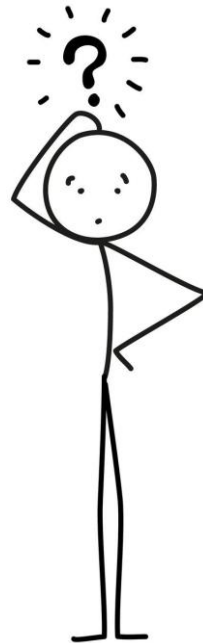
Approximation ratio of Christofides

Theorem (Christofides): If the triangle inequality holds and edge costs are non-negative, then Christofides' algorithm yields a 1.5 - approximation.

Proof:

- Let T be the MST of the given graph G .
- The weight of T , denoted as $w(T)$, is at most $w(T) \leq w(OPT)$
- O is the set of vertices with odd degree in T . The number of odd vertices in G is even, which follows from the Handshaking lemma.
- A perfect matching (M) ensures that the total number of edges incident to each vertex in O is even.
- The weight of this matching $w(M) \leq 0.5 w(OPT)$, because OPT can be decomposed into two matchings. Note that not all vertices in the MST have odd degree.
- Adding the edges of M to T results in a graph $T \cup M$ where all vertices have even degree.
- $T \cup M$ contains an Eulerian circuit because every vertex has an even degree.
- Convert the Eulerian into a Hamiltonian circuit (HAM) by shortcutting repeated vertices.
- $w(HAM) \leq w(T) + w(M) \leq w(OPT) + 0.5w(OPT) = 1.5 w(OPT)$

Sketch the branch-and-bound procedure and consider why initial heuristics can be useful.

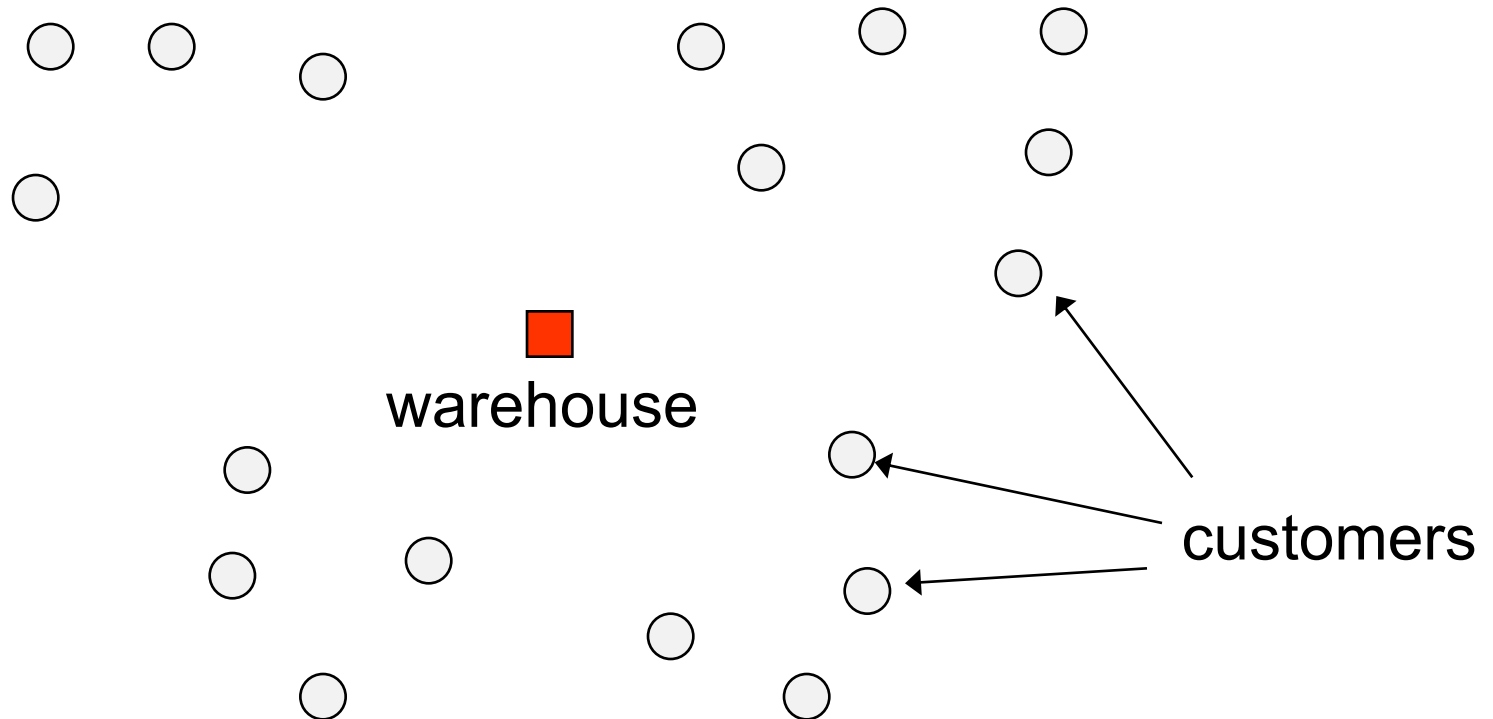


Applications and extensions of TSP

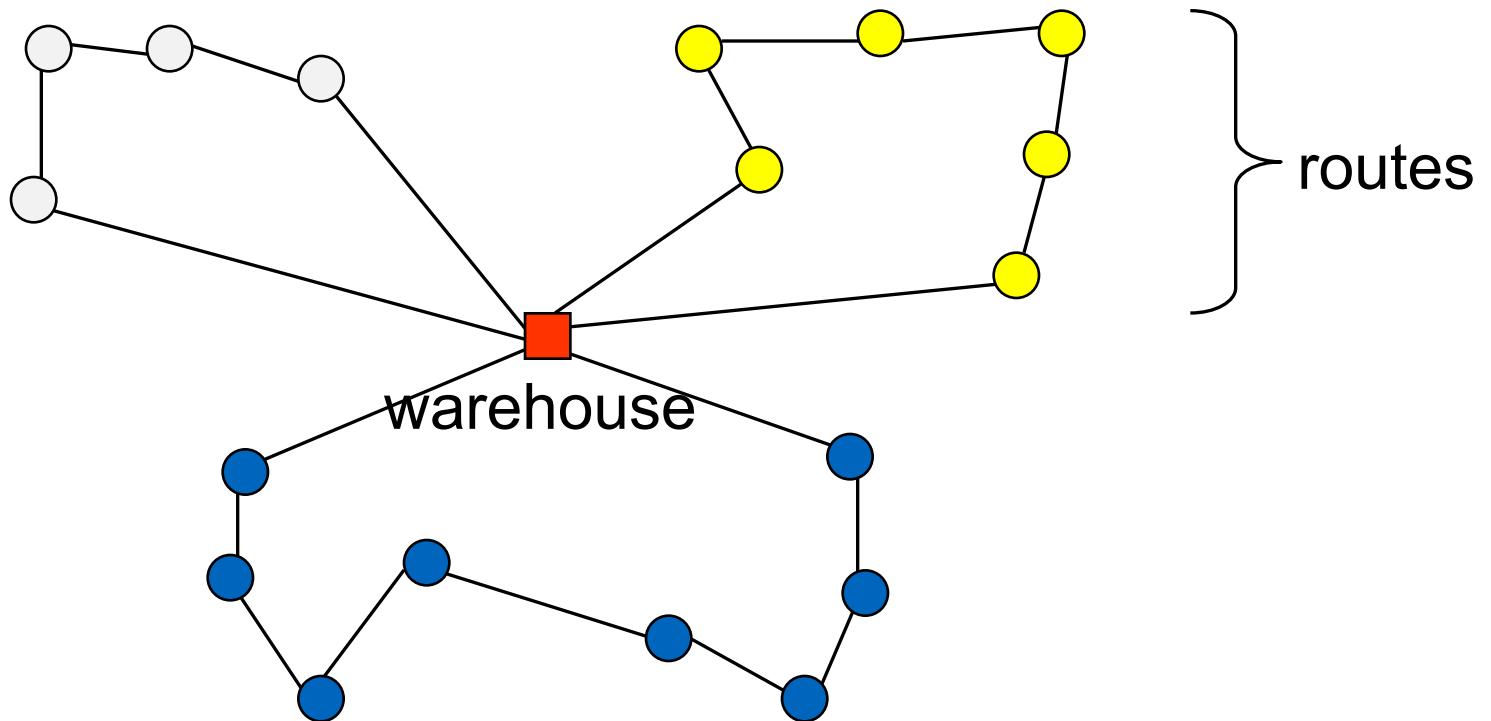
Several problems can be represented as TSP:

- Scheduling
 - A factory produces k products sequentially on a machine.
 - The cost to prepare the machine from producing product i to producing product j is d_{ij} .
 - In what order should the products be produced?
- DNA sequencing
- Layout of integrated circuits
- Vehicle Routing Problems (VRP) as an extension
 - VRPs contain multiple traveling salesmen (vehicles)
- Etc.

The Vehicle Routing Problem (VRP)



The Vehicle Routing Problem (VRP)



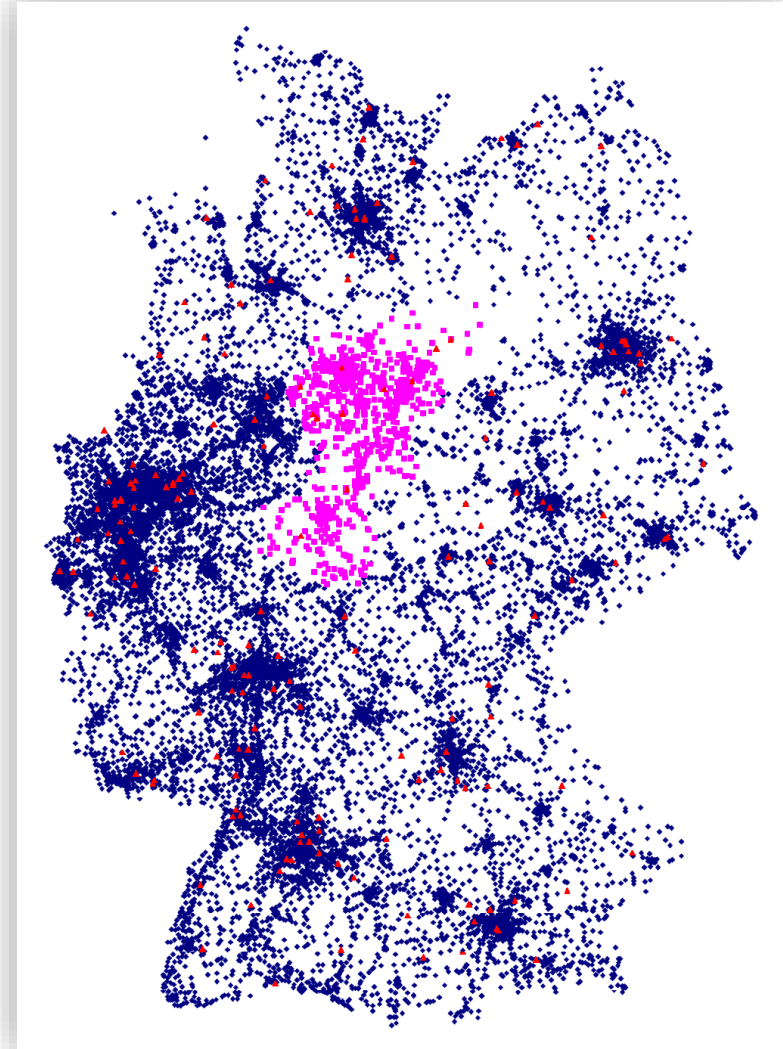
Types of VRPs

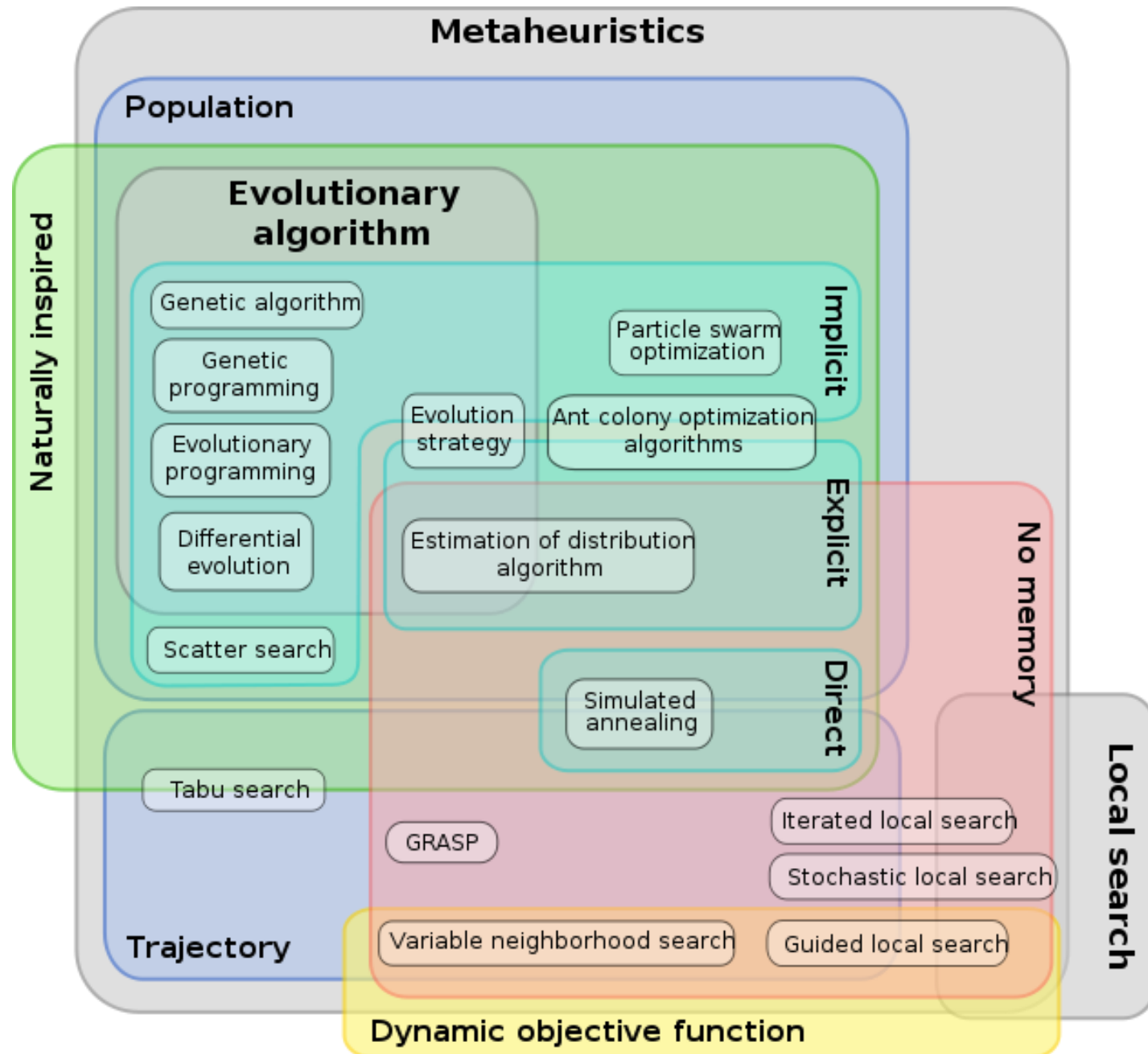
- Each vehicle has limited capacity (Capacitated VRP - CVRP).
- Each customer must be supplied in a certain time-slot (VRP with time windows - VRPTW).
- There are several depots from which the vehicles depart (Multiple Depot VRP - MDVRP).
- Vehicles pick up and deliver cargo (in specific time-slots) (VRP with Pick-Up and Delivering - VRPPD).
- VRPs, in which new requirements arise dynamically (Dynamic VRP – DVRP).
- Vehicles are responsible to fulfill the demand at each point (Inventory Routing Problem - IRP).

Example from practice: route planning of field technicians

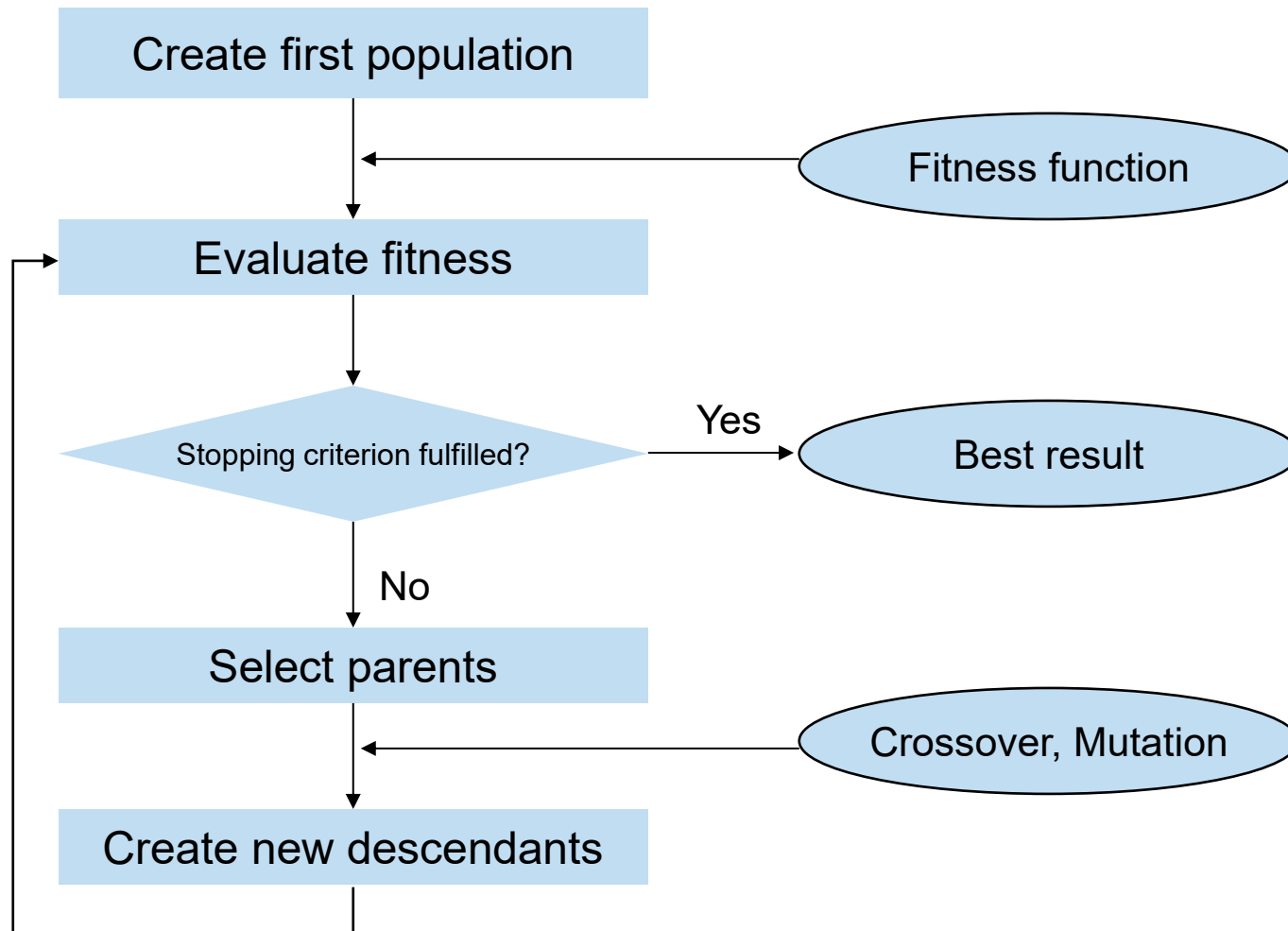
- optimizing deployment of more than 200 field technicians throughout Germany
- coordinating maintenance and fault clearance tasks at spatially separated mobile communications base stations
- increase in productivity
 - ratio of travel time / processing time
 - minimize waiting time
 - reasonable overtime hours
- best use of technicians' skills
- compliance with service levels
- balanced utilization of technicians

The complexity of such problems often leads to the use of **metaheuristics**.





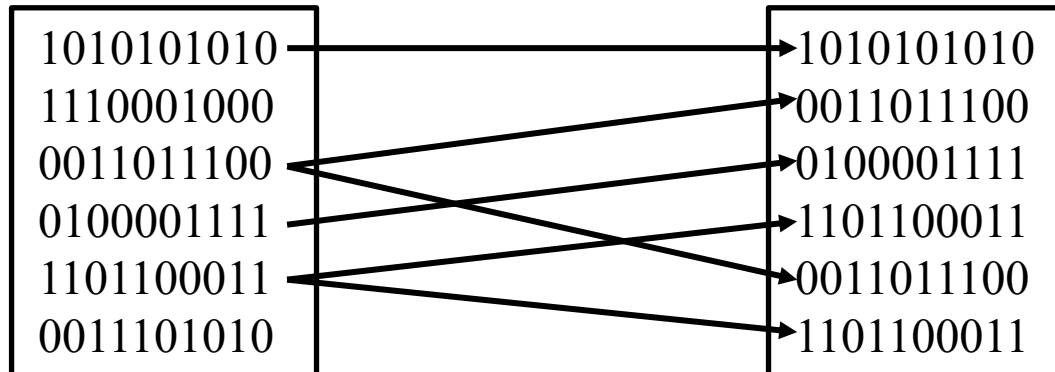
Evolutionary computation: general scheme



Operators (for bit strings)

1. Selection:

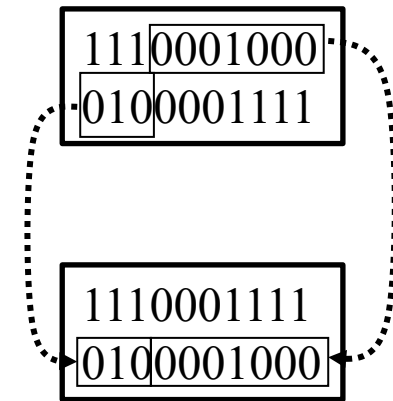
- random sequences of cities in a TSP represent individual solutions (strings)
- strings are copied according to the value of the fitness function
- artificial version of natural selection



Operators

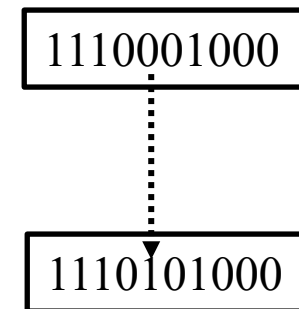
2. Crossover plus recombination:

- Newly produced strings are paired randomly. In this process, parts of the information of the two selected strings are exchanged.
- For example, the partial sequence of cities 6-7-5 is selected from one string, the cities are deleted from another sequence, and appended (in any order) to the second string.



3. Mutation:

- Swapping "1" to "0" or vice versa at random locations. This helps to avoid local optima.
- In TSP, for example, partial sequences of cities would be permuted in a string.



Variants of evolutionary computation

Evolutionary Programming (EP)

- [L. Fogel et al., 1966]
- Only mutation, typically no crossover/recombination

Evolution Strategy (ES)

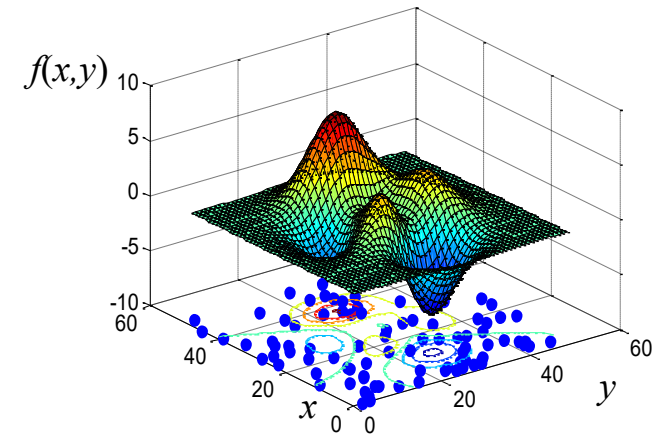
- [I. Rechenberg/Schwefel, 1960s-1970s]
- Real-valued vectors, mutation and recombination

Genetic Algorithm (GA)

- [J. Holland, 1975]
- Bitstrings with fixed-length encodings, crossover and mutation, fitness-based selection

Genetic Programming (GP)

- [J. Koza, 1992]
- Tree representation, crossover and mutation, selection often fitness-based



No Free Lunch theorem by D. H. Wolpert and Macready (1997):

„There exists no universally good method for solving an optimization problem, when the set of *all* problems is considered. If a certain strategy is better in one subdomain than another, it must be worse in another subdomain.“

Evolutionary programming

```
procedure Evo_Programming
{
    t = 0;
    initialize population P(t);
    evaluate P(t);
    until (done) {
        t = t + 1;
        parent_selection P(t);
        mutate P(t);
        evaluate P(t);
        survive P(t);
    }
}
```

All N individuals are **selected** as parents and then **mutated** to produce N children.

These children will be evaluated and N "survivors" are selected by "fitness" in a randomized function.

Thus, individuals with greatest fitness have a higher chance of surviving.

Evolutionary strategies

```
procedure Evo_Strategies
{
  t = 0;
  initialize population P(t);
  evaluate P(t);
  until (done) {
    t = t + 1;
    parent_selection P(t);
    crossover/recombine P(t);
    mutate P(t);
    evaluate P(t);
    survive P(t);
  }
}
```

Individuals (or candidates) are represented as **real-valued vectors**.

After initialization and evaluation, random individuals are **selected** as parents.

In the standard implementation, pairs of parents produce children via **recombination**, which in turn are modified via **mutation**. Mutation is more important, and so is step-size control.

The number of children is greater than N.

Survival rates are deterministic and implemented in two ways:

- The N "fittest" children survive and replace the parents.
- The N "fittest" children and their parents survive.

Genetic algorithms

```
procedure Genetic_Algorithm
{
    t = 0;
    initialize population P(t);
    evaluate P(t);
    until (done) {
        t = t + 1;
        parent_selection P(t);
        crossover/recombine P(t);
        mutate P(t);
        evaluate P(t);
        survive P(t);
    }
}
```

Genetic algorithms are based on a representation of individuals as a **binary string**.

After initialization, parents are selected with a randomized function **based on relative fitness**. I.e., the individuals with higher relative fitness are selected as parents with higher probability.

N children are selected via **recombination** from the N parents.

The N children are **mutated**. They then replace the parents in the population.

Characteristic of genetic algorithms is, besides mutation, the **crossover**.

Variant:

Genetic programs use more complex data structures (binary trees) than GAs.

Metaheuristics and the TSP

- Genetic algorithms can be used for the TSP, where a tour is a permutation of cities, e.g. 1-4-3-2-5-1.
- Start generating random feasible tours.
- Select better tours with higher probability. Shorter tours have higher fitness.
- Crossover/recombination: Combine parts of two parent tours while preserving a valid permutation (each city appears only once).
- Mutation: Swap two cities, reserve a subsequence, or move one city elsewhere.
- Keep the best offspring.
- Stop after a fixed number of generations or no/little improvement.

A pure GA is usually not the best-performing TSP heuristic. Typically, GAs are combined with local search such as 2-opt or variable neighborhood search.

V · T · E

Optimization: Algorithms, methods, and heuristics

[hide]

Unconstrained nonlinear

[hide]

Functions	Golden-section search · Interpolation methods · Line search · Nelder–Mead method · Successive parabolic interpolation	
Gradients	Convergence	Trust region · Wolfe conditions
	Quasi–Newton	Berndt–Hall–Hall–Hausman · Broyden–Fletcher–Goldfarb–Shanno and L-BFGS · Davidon–Fletcher–Powell · Symmetric rank-one (SR1)
	Other methods	Gauss–Newton · Gradient · Levenberg–Marquardt · Conjugate gradient · Truncated Newton
Hessians	Newton's method	

Constrained nonlinear

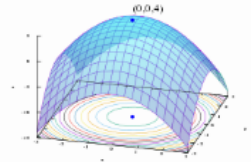
[hide]

General	Barrier methods · Penalty methods
Differentiable	Augmented Lagrangian methods · Sequential quadratic programming · Successive linear programming

Convex optimization

[hide]

Convex minimization	Cutting-plane method · Reduced gradient (Frank–Wolfe) · Subgradient method	
Linear and quadratic	Interior point	Affine scaling · Ellipsoid algorithm of Khachiyan · Projective algorithm of Karmarkar
	Basis-exchange	Simplex algorithm of Dantzig · Revised simplex algorithm · Criss-cross algorithm · Principal pivoting algorithm of Lemke



Combinatorial

[hide]

Paradigms	Approximation algorithm · Dynamic programming · Greedy algorithm · Integer programming (Branch and bound/cut)	
Graph algorithms	Minimum spanning tree	Borůvka · Prim · Kruskal
	Bellman–Ford · Dijkstra · Floyd–Warshall	
Network flows	Dinic · Edmonds–Karp · Ford–Fulkerson · Push–relabel maximum flow	

Metaheuristics

[hide]

Evolutionary algorithm · Hill climbing · Local search · Simulated annealing · Tabu search